

Gas Optimisation Report

April 24, 2026

1 Introduction

Gas optimisation is important in Solidity because storage writes, external calls, dynamic memory operations, and unnecessary bookkeeping all increase execution cost. This report compares a less efficient baseline implementation with an improved implementation for the most expensive function in each contract, using Foundry's `forge test --gas-report`.

2 Contract Comparison: IdentityRegistry.sol

Function Optimised: registerIdentity

```
address[] private _registeredUsers;  
for (uint256 i = 0; i < _registeredUsers.  
length; i++) {  
    require(_registeredUsers[i] != msg.sender,  
        "Already registered");  
}  
_registeredUsers.push(msg.sender);
```

Listing 1: Baseline: linear duplicate scan

```
require(_identities[msg.sender].status ==  
    Status.NotRegistered, "Already registered"  
);  
// No auxiliary user array push
```

Listing 2: Optimized: direct mapping status check

src/IdentityRegistry.sol:IdentityRegistry Contract						
Deployment Cost		Deployment Size				
1638836		7996				
Function Name	Min	Avg	Median	Max	# Calls	
VERIFIER_ROLE	435	435	435	435	2	
addVerifier	24440	38804	38806	53165	4	
getDocumentHash	3135	4142	4142	5150	2	
getIdentity	8469	8469	8469	8469	3	
getStatus	3031	3031	3031	3031	5	
hasRole	3167	3167	3167	3167	2	
isVerified	2956	2956	2956	2956	275	
registerIdentity	21971	114952	116201	116201	299	
removeVerifier	24473	26751	24480	31300	3	
revokeIdentity	24470	30512	27023	36932	7	
verifyIdentity	24513	54273	54668	54668	293	

Figure 1: Baseline Gas Consumption

src/IdentityRegistry.sol:IdentityRegistry Contract						
Deployment Cost		Deployment Size				
1591369		7775				
Function Name	Min	Avg	Median	Max	# Calls	
VERIFIER_ROLE	435	435	435	435	2	
addVerifier	24440	38804	38806	53165	4	
getDocumentHash	3135	4142	4142	5150	2	
getIdentity	8469	8469	8469	8469	3	
getStatus	3031	3031	3031	3031	5	
hasRole	3167	3167	3167	3167	2	
isVerified	2956	2956	2956	2956	275	
registerIdentity	21971	71576	71902	71902	299	
removeVerifier	24473	26751	24480	31300	3	
revokeIdentity	24470	30512	27023	36932	7	
verifyIdentity	24513	54273	54668	54668	293	

Figure 2: Optimized Gas Consumption

Table 1: Efficiency Gains and Analysis

Function	Old Gas	New Gas	Reduction	Commentary
registerIdentity	114,952	71,576	-37.7%	The baseline scanned an auxiliary array before each registration. The optimized version checks registration status directly from the mapping, significantly lowering storage and compute costs.

3 Contract Comparison: KYCGatedAuction.sol

Function Optimised: placeBid

```
struct BidRecord { address bidder; uint256
    amount; }
BidRecord[] private _bidHistory;
_bidHistory.push(BidRecord({ bidder: msg.
    sender, amount: msg.value }));
```

Listing 3: Baseline: unnecessary on-chain bid history

```
// Removed on-chain bid history storage
highestBidder = msg.sender;
highestBid = msg.value;
emit BidPlaced(msg.sender, msg.value);
```

Listing 4: Optimized: rely on state and events only

src/KYCGatedAuction.sol:KYCGatedAuction Contract					
Deployment Cost	Deployment Size				
1210878	6387				
Function Name	Min	Avg	Median	Max	# Calls
endAuction	23837	44054	51706	51706	7
ended	2489	2489	2489	2489	1
highestBid	2469	2469	2469	2469	259
highestBidder	2530	2530	2530	2530	3
pendingReturns	2869	2869	2869	2869	1
placeBid	28982	142516	144942	144942	273
withdrawProceeds	29601	42240	31777	65342	3
withdrawRefund	29005	31013	31013	33021	2

Figure 3: Baseline Gas Consumption

src/KYCGatedAuction.sol:KYCGatedAuction Contract					
Deployment Cost	Deployment Size				
1175639	6224				
Function Name	Min	Avg	Median	Max	# Calls
endAuction	23837	44054	51706	51706	7
ended	2489	2489	2489	2489	1
highestBid	2469	2469	2469	2469	259
highestBidder	2530	2530	2530	2530	3
pendingReturns	2869	2869	2869	2869	1
placeBid	28982	77394	78350	78350	273
withdrawProceeds	29601	42240	31777	65342	3
withdrawRefund	29005	31013	31013	33021	2

Figure 4: Optimized Gas Consumption

Table 2: Efficiency Gains and Analysis

Function	Old Gas	New Gas	Reduction	Commentary
placeBid	142,516	77394	-45.7%	The baseline stored unnecessary bid history on-chain. The optimized version removes this redundant tracking and relies on state variables and events, significantly reducing gas.

4 Conclusion

The gas report shows that targeted optimisations can reduce execution cost by removing redundant state writes, avoiding unnecessary external calls, and replacing auxiliary bookkeeping with direct storage checks. The largest improvements were observed in `setShouldVerify` and `placeBid`, where unnecessary on-chain history tracking was removed.